

Nesterov's Smooth Minimization of Non-Smooth Functions

Theory, implementation, and computational experiments

Michał Szewczak Sebastian Trojan

June 20, 2026

Based on Yurii Nesterov, *Smooth minimization of non-smooth functions*, 2005.

Many convex optimization problems are non-smooth because their objectives contain maxima, norms, or absolute values.

Classical subgradient methods for non-smooth convex optimization have a worst-case rate of order

$$O(1/\varepsilon^2).$$

Nesterov's smoothing method uses the max-structure of the objective to construct a smooth approximation. After smoothing, an accelerated first-order method can be applied, leading to the improved complexity

$$O(1/\varepsilon).$$

The price is a trade-off: a smaller smoothing parameter gives a more accurate approximation, but a harder smooth problem.

Structured problem class

The original problem is

$$f^* = \min_{x \in Q_1} f(x),$$

where

$$f(x) = \hat{f}(x) + \max_{u \in Q_2} \{\langle Ax, u \rangle - \hat{\phi}(u)\}.$$

- x is the primal variable.
- u is the dual variable.
- $\hat{f}(x)$ is the smooth part of the objective.
- In the implemented examples, $\hat{f}(x) = 0$.
- The non-smoothness comes from the maximization over u .

Smoothing the max-term

Choose a strongly convex prox-function d_2 on Q_2 and define

$$f_\mu(x) = \max_{u \in Q_2} \{ \langle Ax, u \rangle - \hat{\phi}(u) - \mu d_2(u) \}.$$

The full smoothed objective is

$$\bar{f}_\mu(x) = \hat{f}(x) + f_\mu(x).$$

The approximation error is bounded by

$$\bar{f}_\mu(x) \leq f(x) \leq \bar{f}_\mu(x) + \mu D_2.$$

Smaller μ gives a more accurate approximation, but the smoothed problem becomes harder.

Gradient and smoothness trade-off

The smoothed dual response is

$$u_\mu(x) = \arg \max_{u \in Q_2} \{ \langle Ax, u \rangle - \hat{\phi}(u) - \mu d_2(u) \}.$$

Then

$$\nabla f_\mu(x) = A^* u_\mu(x), \quad \nabla \bar{f}_\mu(x) = \nabla \hat{f}(x) + A^* u_\mu(x).$$

The gradient is Lipschitz continuous with constant

$$L_\mu = M + \frac{\|A\|_{1,2}^2}{\mu \sigma_2}.$$

This formula explains the central trade-off: decreasing μ improves the approximation but increases L_μ .

Accelerated minimization

For a fixed smoothing parameter μ , the method minimizes

$$\min_{x \in Q_1} \bar{f}_\mu(x).$$

One accelerated iteration

g_k	$=$	$\nabla \bar{f}_\mu(x_k)$	gradient,
y_k	$=$	$T_{Q_1}(x_k)$	local point,
G_k	$=$	$\sum_{i=0}^k \alpha_i g_i$	accumulated gradient,
z_k	$=$	$\arg \min_{x \in Q_1} \left\{ \frac{L}{\sigma_1} d_1(x) + \langle G_k, x \rangle \right\}$	aggregate point,
x_{k+1}	$=$	$\frac{2}{k+3} z_k + \frac{k+1}{k+3} y_k$	next search point.

The local point is obtained from the quadratic model

$$T_{Q_1}(x_k) = \arg \min_{y \in Q_1} \left\{ \langle g_k, y - x_k \rangle + \frac{L}{2} \|y - x_k\|^2 \right\}.$$

Primal-dual certificate and complexity

The algorithm builds an averaged dual point

$$\hat{u} = \frac{\sum_{k=0}^N \alpha_k u_k}{\sum_{k=0}^N \alpha_k}, \quad u_k = u_\mu(x_k),$$

and uses $\hat{x} = y_N$ as the final primal point.

By weak duality,

$$\phi(\hat{u}) \leq f^* \leq f(\hat{x}),$$

so the primal-dual gap

$$f(\hat{x}) - \phi(\hat{u})$$

certifies the solution quality.

For the implemented examples, $M = 0$, and the theoretical iteration complexity is

$$N = O(1/\varepsilon), \quad \mu = \frac{\varepsilon}{2D_2}.$$

Matrix game: $\min_{x \in \Delta_n} \max_{u \in \Delta_m} u^\top A x$

Continuous location: $\min_{\|x\|_2 \leq r} \sum_{j=1}^p w_j \|x - c_j\|_2$

Maximum of absolute values: $\min_{\|x\|_2 \leq r} \max_{1 \leq j \leq m} |a_j^\top x - b_j|$

Sum of absolute values: $\min_{\|x\|_2 \leq r} \sum_{j=1}^m |a_j^\top x - b_j|$

Experimental plan

- 1 Reproduce Nesterov's matrix game experiment.
- 2 Apply the same scheme to the other three examples.
- 3 Test the empirical dependence on ε .
- 4 Compare standard smoothing with a changing- μ strategy.

In every experiment, convergence is measured by the certified condition

$$f(\hat{x}) - \phi(\hat{u}) \leq \varepsilon.$$

Experiment 1: matrix game reproduction

Reproduced problem:

$$\min_{x \in \Delta_n} \max_{u \in \Delta_m} u^\top A x, \quad A_{ij} \sim U[-1, 1].$$

Tested grid:

- $\varepsilon \in \{10^{-2}, 10^{-3}, 10^{-4}\},$
- $m \in \{100, 300, 1000\},$
- $n \in \{100, 300, 1000, 3000, 10000\},$ with the largest n omitted for $\varepsilon = 10^{-4}.$

Each table cell reports iterations / percentage of the predicted iteration bound.

Experiment 1: matrix game results

$\epsilon = 10^{-2}$					
$m \backslash n$	100	300	1000	3000	10000
100	765/42%	934/46%	1108/49%	1187/49%	1281/49%
300	690/34%	910/40%	1126/45%	1268/47%	1475/51%
1000	761/34%	873/35%	1086/39%	1284/43%	1457/46%
$\epsilon = 10^{-3}$					
$m \backslash n$	100	300	1000	3000	10000
100	7123/39%	8426/41%	8928/40%	10191/42%	10080/39%
300	7337/36%	8639/38%	10154/40%	11302/42%	12022/41%
1000	7526/33%	8828/35%	10157/37%	11595/39%	14167/44%
$\epsilon = 10^{-4}$					
$m \backslash n$	100	300	1000	3000	
100	68505/37%	64512/31%	77136/34%	74024/30%	
300	60932/30%	78680/34%	83844/33%	92862/34%	
1000	70778/31%	77644/31%	93346/34%	102925/35%	

Experiment 2: other problem classes

The matrix game experiment was extended to:

- continuous location,
- maximum of absolute values,
- sum of absolute values.

In each case the same type of information was recorded:

- number of iterations,
- percentage of the theoretical prediction,
- final primal-dual gap,
- runtime.

Experiment 2: summary tables

Continuous location				
ϵ	Avg. iter.	Max iter.	Avg. %	Time
10^{-4}	171599	320867	30.64%	31.8s
10^{-3}	20525	31653	43.74%	3.86s
10^{-2}	1988	3606	36.70%	0.35s
Maximum of absolute values				
ϵ	Avg. iter.	Max iter.	Avg. %	Time
10^{-4}	171486	249861	39.81%	10.6s
10^{-3}	20032	33671	44.76%	1.26s
10^{-2}	2593	3788	56.06%	0.16s
Sum of absolute values				
ϵ	Avg. iter.	Max iter.	Avg. %	Time
10^{-4}	1198404	2711368	29.45%	54.9s
10^{-3}	135894	284262	32.52%	6.43s
10^{-2}	12088	27750	30.43%	0.51s

Experiment 3: testing the $O(1/\varepsilon)$ behavior

To isolate the dependence on ε , one random instance was fixed for each problem class and solved for

$$\varepsilon \in \{10^{-1}, 5 \cdot 10^{-2}, 10^{-2}, 5 \cdot 10^{-3}, 10^{-3}, 5 \cdot 10^{-4}, 10^{-4}\}.$$

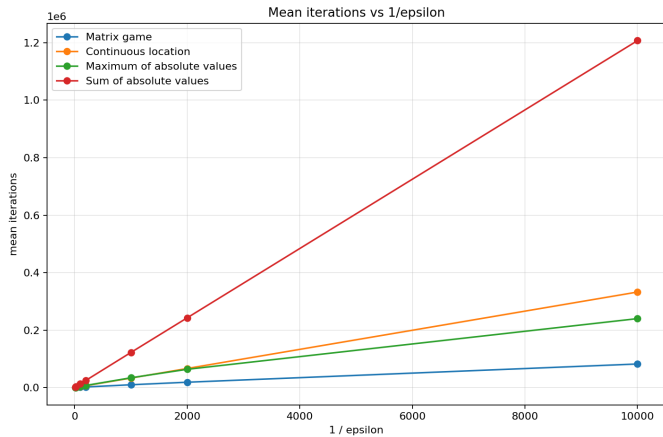
The theory predicts

$$N(\varepsilon) \approx C \frac{1}{\varepsilon}.$$

Therefore:

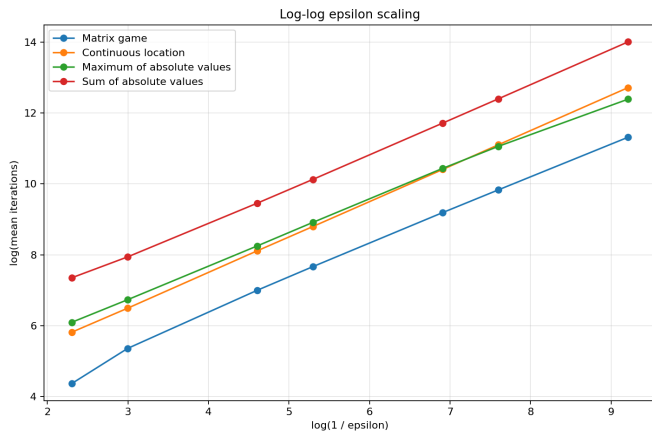
- iterations versus $1/\varepsilon$ should be approximately linear,
- the log-log fitted exponent $\hat{p}$ should be close to 1.

Experiment 3: iterations versus $1/\varepsilon$



The curves are close to linear; the difference between problems is mainly the constant factor.

Experiment 3: log-log scaling



Empirical power-law model:

$$N(\epsilon) \approx C\epsilon^{-p}.$$

Taking logarithms gives

$$\log N = \log C + p \log(1/\epsilon).$$

Therefore $\hat{p}$ is the slope of the fitted line on the log-log plot. The theory predicts

$$p = 1.$$

Problem	$\hat{p}$	SE
Matrix game	0.9897	0.0096
Continuous location	0.9986	0.0054
Max. abs. values	0.9208	0.0042
Sum abs. values	0.9649	0.0096

Experiment 4: changing the smoothing parameter

1. Motivation

- Standard smoothing uses $\mu_\varepsilon = \varepsilon/(2D_2)$ from the beginning.
- For small ε , this value is small, so the smoothed problem is sharper and harder.

2. Method

- Start with a larger smoothing parameter and decrease it geometrically:

$$\mu_0 = c\mu_\varepsilon, \quad \mu_{t+1} = \rho\mu_t.$$

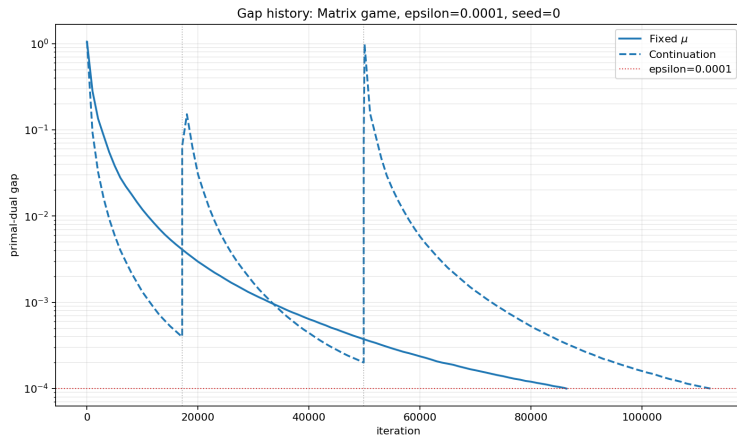
- Move to the next stage when

$$f(\hat{x}) - \phi(\hat{u}) \leq c_{\text{stage}}\mu_t D_2.$$

3. Our parameters

$$c = 8, \quad \rho = 0.5, \quad c_{\text{stage}} = 1, \quad 8\mu_\varepsilon \rightarrow 4\mu_\varepsilon \rightarrow 2\mu_\varepsilon \rightarrow \mu_\varepsilon.$$

Experiment 4: gap history



After reducing μ , the smoothed objective changes and the primal-dual gap can jump upward. In most tested cases, continuation required more iterations.

- The implementation reached the required primal-dual gap on all reported instances.
- Matrix game results reproduce the qualitative behavior from Nesterov's paper.
- The empirical scaling is consistent with the theoretical $O(1/\varepsilon)$ complexity.
- Worst-case iteration bounds were conservative on random instances.
- Practical difficulty depends strongly on the problem structure.
- The tested changing- μ strategy was usually less efficient than the standard smoothing method.

-  [Yurii Nesterov.](#)
Smooth minimization of non-smooth functions.
[Mathematical Programming](#), 103(1):127–152, 2005.